

External Dependencies for Typescript

References

- [jqassistant](#)
- [Neo4j Python Driver](#)

External Typescript Module Usage

External Module

An external Typescript module is marked with the label `ExternalModule` and the declarations it provides with `ExternalDeclaration`. In practice, the distinction between internal and external isn't always that clear. When there is a problem following the project configuration like discussed in [Missing Interfaces and other elements in the Graph](#), some internal dependencies might be imported as external ones.

To have a second indicator, the property `isNodeModule` is written with [Add_module_properties.cypher](#) in [prepareAnalysis.sh](#). For most package managers this should then be sufficient. As of now (June 2024), it might not work with [Yarn Plug'n'Play](#).

Table 1 - Top 20 most used external packages overall

This table shows the external packages that are used by the most different internal types overall. Additionally, it shows which types of the external modules are actually used. External annotations are also listed.

Only the top 20 entries are shown. The whole table can be found in the following CSV report: `External_module_usage_overall_for_Typescript`

Columns:

- *externalModuleName* is the name of the external module prepended by its namespace if given. Example: "@types/react"

- *numberOfExternalCallerModules* is the number of modules that use that external module
- *numberOfExternalCallerElements* is the number of elements (functions, classes,...) that use that external module
- *numberOfExternalDeclarationCalls* is how often the external declarations of that external module are imported
- *numberOfExternalDeclarationCallsWeighted* is how often the external declarations of that external module are actually used
- *allModules* contains the total count of all analyzed internal modules
- *allInternalElements* contains the total count of all analyzed exported internal elements (function, classes,...)
- *exampleStories* contains a list of sentences that contain concrete examples (for explanation and debugging)

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 4, column: 27, offset: 138} for query: "// External Typescript module usage overall\n\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n WITH count(DISTINCT internalElement.globalFqn) AS allInternalElements\n ,count(DISTINCT internalModule.globalFqn) AS allModules\n ,collect(DISTINCT internalElement) AS internalElementList\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[externalDependency:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH allInternalElements\n ,allModules\n ,coalesce(nullIf(externalModule.namespace, '') + '/' + externalModule.name, externalModule.name) AS externalModuleName\n ,count(DISTINCT internalModule.globalFqn) AS numberOfExternalCallerModules\n ,count(DISTINCT internalElement.globalFqn) AS numberOfExternalCallerElements\n ,count(externalDependency) AS numberOfExternalDeclarationCalls\n ,sum(externalDependency.cardinality) AS numberOfExternalDeclarationCallsWeighted\n ,collect('<' + internalElement.name\n + '> of module <' + internalModule.name\n + '> imports <' + externalDeclaration.name\n + '> from external module <' + externalModule.name + '>')[0..4] AS exampleStories\n RETURN externalModuleName\n ,numberOfExternalCallerModules\n ,numberOfExternalCallerElements\n ,numberOfExternalDeclarationCalls\n ,numberOfExternalDeclarationCallsWeighted\n ,allModules\n ,allInternalElements\n ,exampleStories\n ORDER BY numberOfExternalCallerModules DESC, externalModuleName ASC"
```

	externalModuleName	numberOfExternalCallerModules	numberOfExternalCallerElements	numberOfExternalDeclarationCalls	numberOfExternalDeclarationCallsWeighted
0	@remix-run/router	4	37	195	
1	@types/react	4	34	97	
2	@types/react-native	1	5	14	
3	@ungap/url-search-params	1	2	2	

Table 1 Chart 1a - Most called external modules in % by internal elements (more than 0.7% overall)

External modules that are used less than 0.7% are grouped into "others" to get a cleaner chart containing the most significant external modules and how often they are called by internal elements in percent.

<Figure size 640x480 with 0 Axes>

Top external module usage [%] by internal elements (more than 0.7% overall)

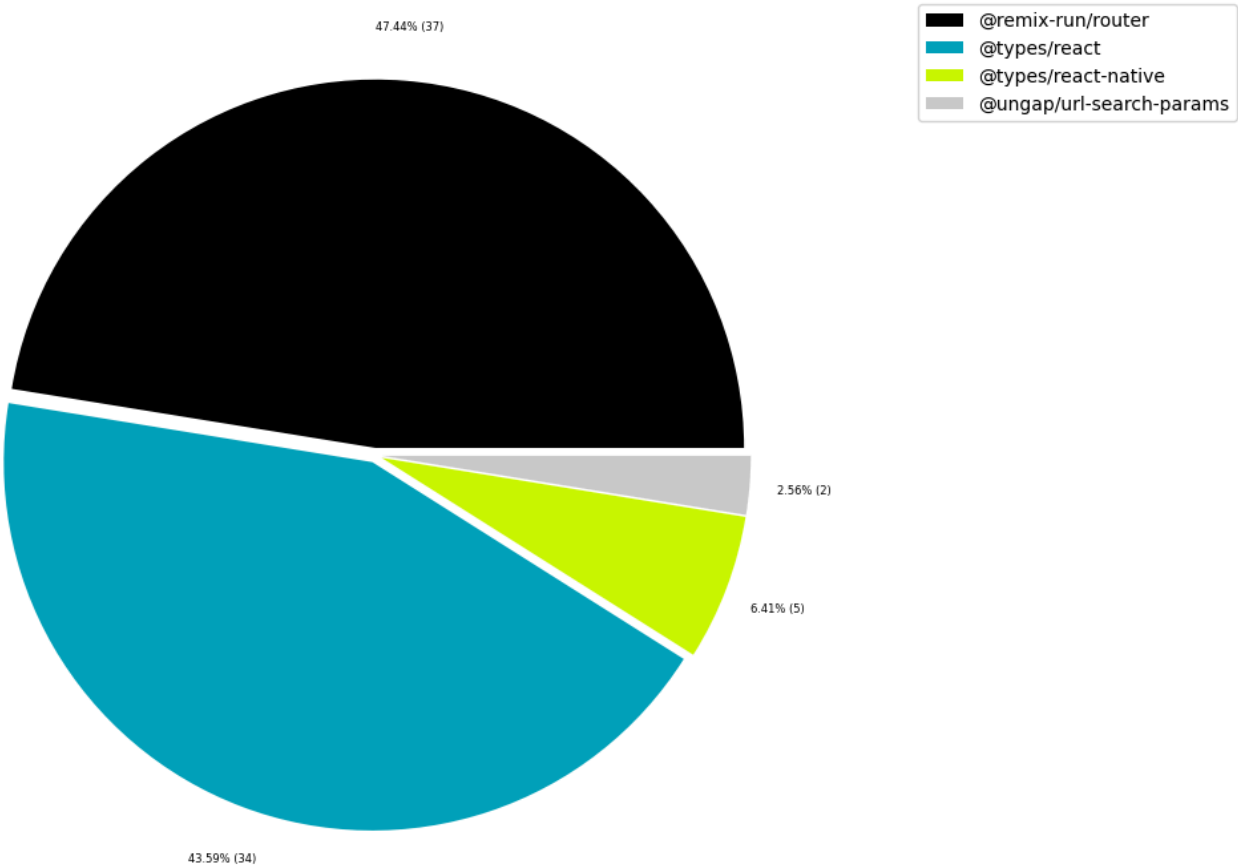


Table 1 Chart 1b - Most called external modules in % by internal elements (less than 0.7% overall "others" drill-down)

Shows the lowest (less than 0.7% overall) most called external modules. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external module usage [%] by internal elements (less than 0.7% overall "others" drill-down)'.

Table 1 Chart 2a - Most called external modules in % by internal modules (more than 0.7% overall)

External modules that are used less than 0.7% are grouped into "others" to get a cleaner chart containing the most significant external modules and how often they are called by internal modules in percent.

<Figure size 640x480 with 0 Axes>

Top external module usage [%] by internal modules (more than 0.7% overall)

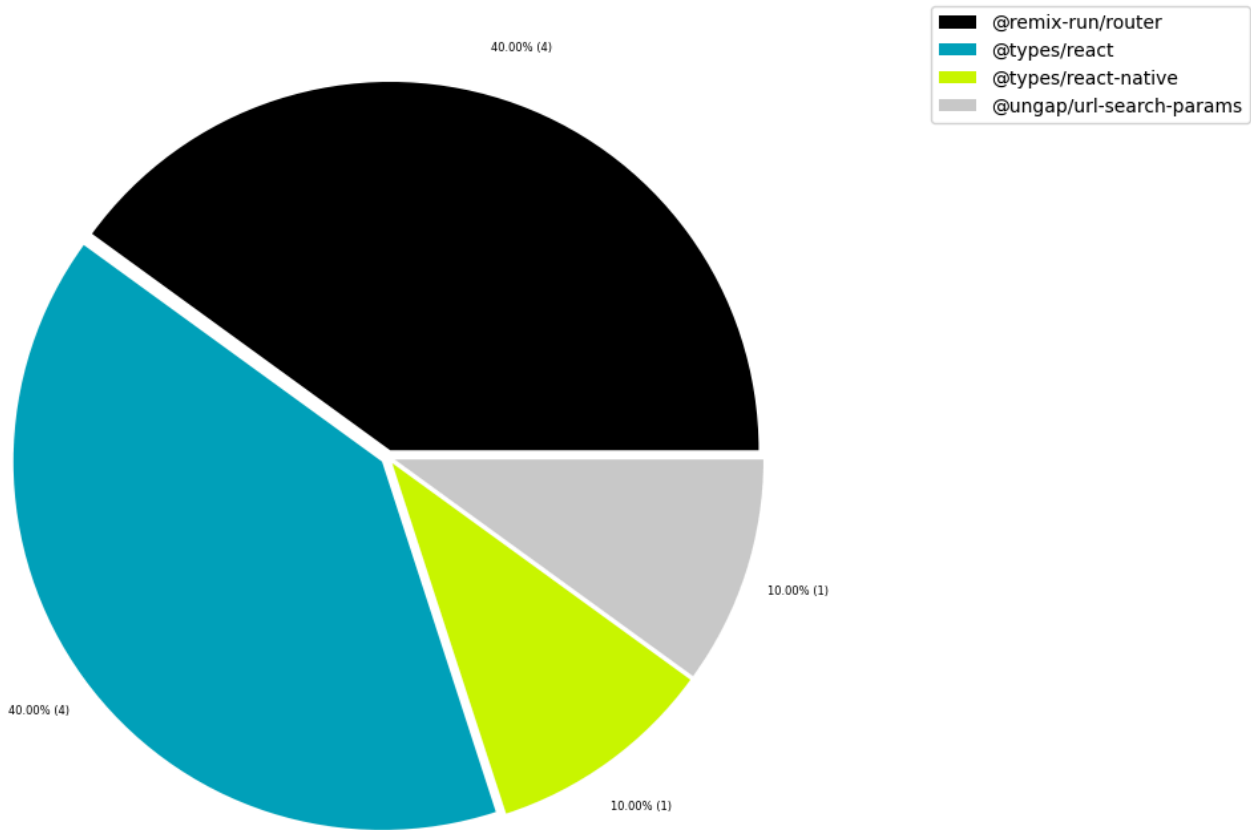


Table 1 Chart 2b - Most called external modules in % by internal modules (less than 0.7% overall "others" drill-down)

Shows the lowest (less than 0.7% overall) most called external modules. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external module usage [%] by internal modules (less than 0.7% overall "others" drill-down)'.

Table 2 - Top 20 most used external namespaces

This table shows external namespaces that are used by the most different internal elements (functions, classes,...) overall.

Additionally, it shows how many of the declarations of the external namespace are actually used.

Only the top 20 entries are shown. The whole table can be found in the following CSV report: `External_namespace_usage_overall_for_Typescript`

Columns:

- *externalNamespaceName* is the name of the external namespace (empty if none). Example: "@types". All other columns are aggregated/grouped by it.
- *numberOfExternalCallerModules* is the number of modules that use that external module
- *numberOfExternalCallerElements* is the number of elements (functions, classes,...) that use that external module
- *numberOfExternalDeclarationCalls* is how often the external declarations of that external module are imported
- *numberOfExternalDeclarationCallsWeighted* is how often the external declarations of that external module are actually used
- *allModules* contains the total count of all analyzed internal modules
- *allInternalElements* contains the total count of all analyzed exported internal elements (function, classes,...)
- *exampleStories* contains a list of sentences that contain concrete examples (for explanation and debugging)

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 4, column: 27, offset: 141} for query: "// External Typescript namespace usage overall\n\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n WITH count(DISTINCT internalElement.globalFqn) AS allInternalElements\n ,count(DISTINCT internalModule.globalFqn) AS allModules\n ,collect(DISTINCT internalElement) AS internalElementList\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[externalDependency:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH allInternalElements\n ,allModules\n ,coalesce(nullif(externalModule.namespace, ''), 'no namespace') AS externalNamespaceName\n ,count(DISTINCT internalModule.globalFqn) AS numberOfExternalCallerModules\n ,count(DISTINCT internalElement.globalFqn) AS numberOfExternalCallerElements\n ,count(externalDependency) AS numberOfExternalDeclarationCalls\n ,sum(externalDependency.cardinality) AS numberOfExternalDeclarationCallsWeighted\n ,collect('<' + internalElement.name\n + internalModule.name\n + '>' imports '<' + externalDeclaration.name\n + '>' from external namespace\n + externalModule.namespace + '>')[0..4] AS exampleStories\n RETURN externalNamespaceName\n ,numberOfExternalCallerModules\n ,numberOfExternalCallerElements\n ,numberOfExternalDeclarationCalls\n ,numberOfExternalDeclarationCallsWeighted\n ,allModules\n ,allInternalElements\n ,exampleStories\n ORDER BY numberOfExternalCallerModules DESC, externalNamespaceName ASC"
```

	externalNamespaceName	numberOfExternalCallerModules	numberOfExternalCallerElements	numberOfExternalDeclarationCalls	numberOfEx
0	@remix-run	4	37	195	
1	@types	4	35	111	
2	@ungap	1	2	2	

Table 2 Chart 1a - Most called external namespaces in % by internal element (more than 0.7% overall)

External namespaces that are used less than 0.7% are grouped into "others" to get a cleaner chart containing the most significant external namespaces and how often they are called by internal elements in percent.

<Figure size 640x480 with 0 Axes>

Top external namespace usage [%] by internal elements (more than 0.7% overall)

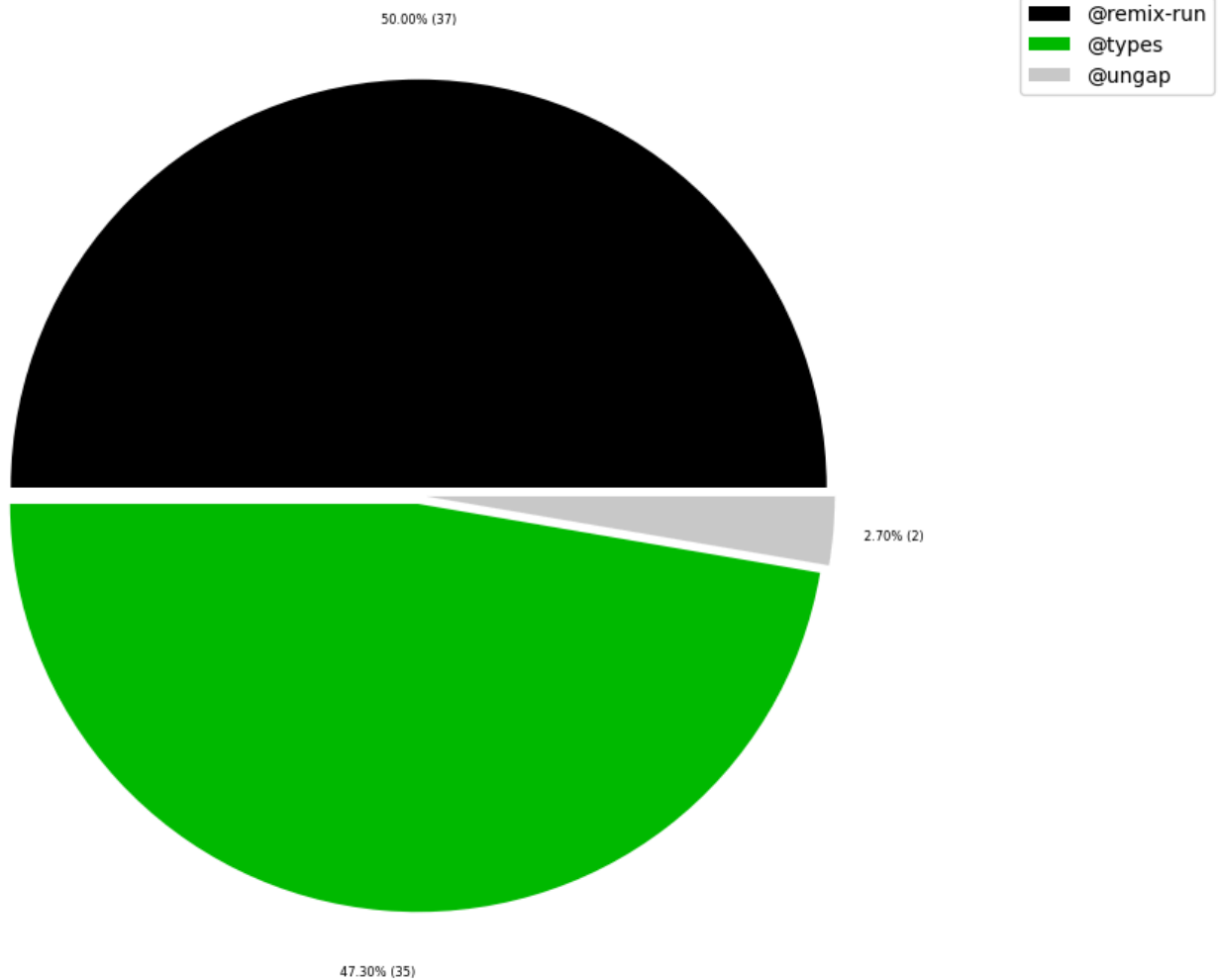


Table 2 Chart 1a - Most called external namespaces in % by internal element (less than 0.7% overall "others" drill-down)

Shows the lowest (less than 0.7% overall) most called external namespaces. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external namespace usage [%] by internal elements (less than 0.7% overall "others" drill-down)'.

Table 2 Chart 2a - Most called external namespaces in % by internal modules (more than 0.7% overall)

External namespaces that are used less than 0.7% are grouped into "others" to get a cleaner chart containing the most significant external namespaces and how often they are called by internal modules in percent.

<Figure size 640x480 with 0 Axes>

Top external namespace usage [%] by internal modules (more than 0.7% overall)

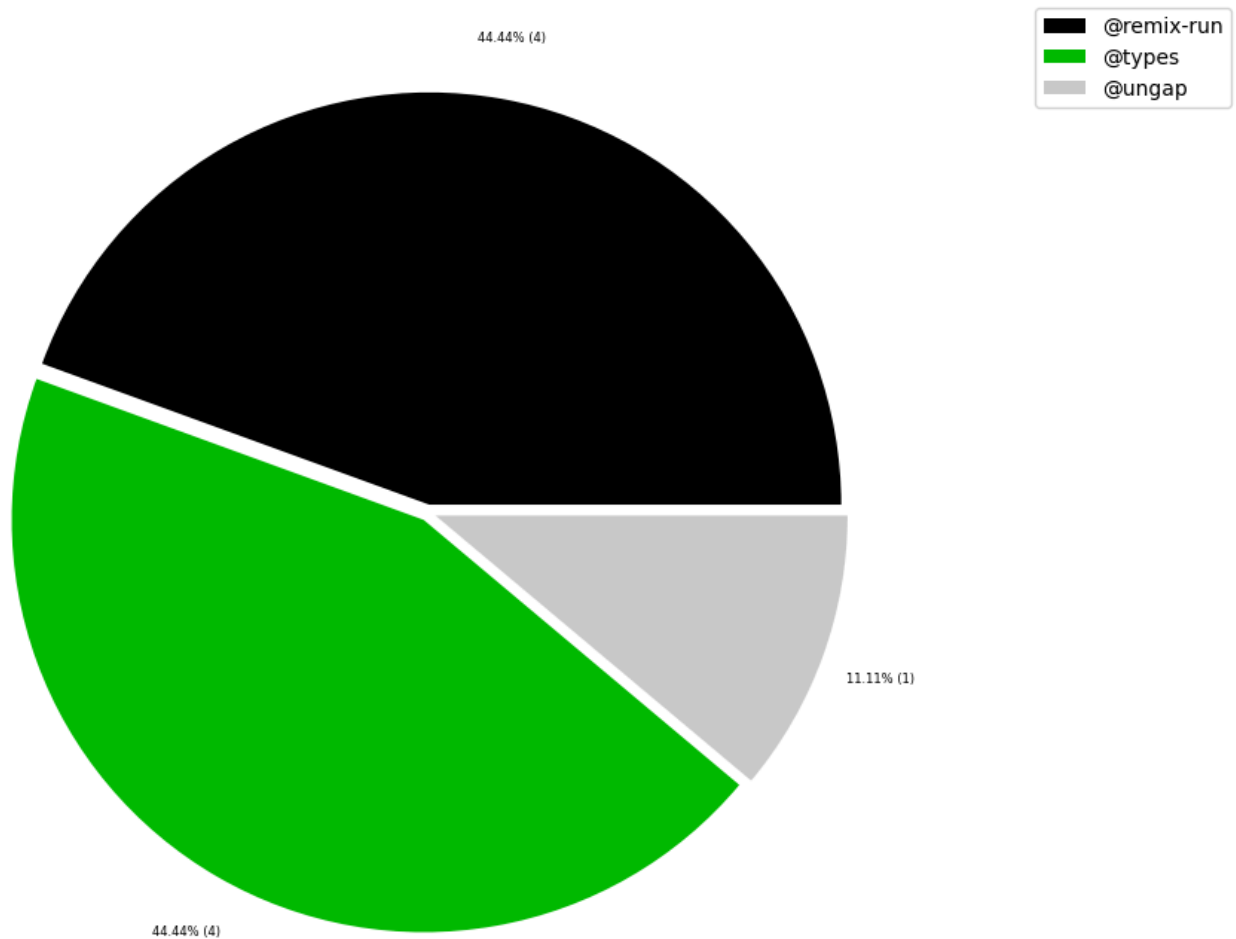


Table 2 Chart 2b - Most called external namespaces in % by internal modules (less than 0.7% overall "others" drill-down)

Shows the lowest (less than 0.7% overall) most called external namespaces. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external namespace usage [%] by internal modules (less than 0.7% overall "others" drill-down)'.

Table 3 - Top 20 most widely spread external modules

The following tables shows external modules that are used by many different internal modules with the highest number of artifacts first.

Statistics like minimum, maximum, average, median and standard deviation are provided for the number of internally exported elements (function, class, ...) and the external declarations they use for every external module.

The intuition behind that is to find external modules and the external declarations they provide that are used in a widely spread manner. This can help to distinguish widely used libraries and frameworks from external modules that are used for specific tasks. It can also be used to find external modules that are used sparsely regarding internal modules but where many different external declarations are used.

Refactoring with [Hexagonal architecture](#) in mind can be considered for non-framework external modules that are used for very specific tasks and that are used in many different internal locations. This makes the internal code more robust against changes of these external modules or it is easier to update and migrate to newer versions of them.

External modules that are only used in very few internal locations overall might be considered for removal if they are easy to replace with a similar library that is already used more often. Or they might also simply be replaced by very few lines of code. Replacing libraries with own code isn't recommended when you need to write a lot of code or for external modules that provide security relevant implementations (encryption, sanitizers, ...), because they will be tracked and maintained globally and security updates need to be adopted fast.

Only the top 20 entries are shown. The whole table can be found in the following CSV report: `External_module_usage_spread_for_Typescript`

Columns:

- *externalModuleName* is the name of the external package prepended by its namespace if given. Example: "@types/react" external package.
- *numberOfInternalModules* is the number of internal modules that are using that external module
- *[min,max,med,avg,std]NumberOfUsedExternalDeclarations* provide statistics for all internal modules and how their usage of the declarations provided by the external

module are distributed. This provides an indicator on how strong the coupling to the external module is. For example, if many (high sum) elements provided by that external module are used constantly (low std), a higher coupling can be presumed. If there is only one (sum) element in use, this could be an indicator for an external module that could get replaced or that there is just one central entry point for it.

- *[min/max/med/avg/std]NumberOfInternalElements* provide statistics for all internal modules and how their usage of the external module is distributed across their internal elements. This provides an indicator on how widely an external module is spread across internal elements and if there are great differences between internal modules (high standard deviation) or not.
- *[min/max/med/avg/std]NumberOfInternalElementsPercentage* is similar to *[min/max/med/avg/std]NumberOfUsedExternalDeclarations* but provides the value in percent in relation to the total number of internal elements per internal module.
- *internalModuleExamples* some examples of included internal modules for debugging

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 5, column: 27, offset: 190} for query: "// External Typescript module usage spread\n\n // Get the overall internal modules statistics first\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n WITH count(DISTINCT internalModule.globalFqn)\n AS internalModulesCountOverall\n ,count(DISTINCT internalElement.globalFqn)\n AS internalElementsCountOverall\n ,collect(DISTINCT internalElement)\n AS internalElementList\n\n // Get the external declarations for each internal element\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:TS:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH internalModulesCountOverall\n ,internalElementsCountOverall\n ,coalesce(nullIf(externalModule.namespace, '') + '/' + externalModule.name, externalModule.name)\n AS externalModuleName\n ,coalesce(nullIf(internalModule.namespace, '') + '/' + internalModule.name, internalModule.name) AS internalModuleName\n\n // Gathering counts for every internal element and the external module it uses\n ,count(DISTINCT externalDeclaration.globalFqn) AS externalDeclarationsCount\n ,COLLECT(DISTINCT externalDeclaration.globalFqn)[0..9] AS externalDeclarationsExamples\n ,count(DISTINCT internalElement.globalFqn) AS internalElementsCount\n ,COLLECT(DISTINCT internalElement.globalFqn)[0..9] AS internalElementsExamples\n ,100.0 / internalModulesCountOverall * count(DISTINCT internalElement.globalFqn) AS internalElementsCallingExternalRate\n\n // Group by external module\n RETURN externalModuleName\n ,count(DISTINCT internalModuleName) AS numberOfInternalModules\n\n // Statistics about how many internal modules are using that external module\n ,sum(internalElementsCount) AS sumNumberOfUsedExternalDeclarations\n ,min(internalElementsCount) AS minNumberOfUsedExternalDeclarations\n ,max(internalElementsCount) AS maxNumberOfUsedExternalDeclarations\n ,percentileCont(internalElementsCount, 0.5) AS medNumberOfUsedExternalDeclarations\n ,avg(internalElementsCount) AS avgNumberOfUsedExternalDeclarations\n ,stDev(internalElementsCount) AS stdNumberOfUsedExternalDeclarations\n\n // Statistics about the internal elements and their external module usage\n ,sum(internalElementsCount) AS sumNumberOfInternalElements\n ,min(internalElementsCount) AS minNumberOfInternalElements\n ,max(internalElementsCount) AS maxNumberOfInternalElements\n ,percentileCont(internalElementsCount, 0.5) AS medNumberOfInternalElements\n ,avg(internalElementsCount) AS avgNumberOfInternalElements\n ,stDev(internalElementsCount) AS stdNumberOfInternalElements\n\n // Statistics about the types and their external package usage count percentage\n ,min(internalElementsCallingExternalRate) AS minNumberOfInternalElementsPercentage\n ,max(internalElementsCallingExternalRate) AS maxNumberOfInternalElementsPercentage\n ,percentileCont(internalElementsCallingExternalRate, 0.5) AS medNumberOfInternalElementsPercentage\n ,avg(internalElementsCallingExternalRate) AS avgNumberOfInternalElementsPercentage\n ,stDev(internalElementsCallingExternalRate) AS stdNumberOfInternalElementsPercentage\n\n ,collect(DISTINCT internalModuleName)[0..4] AS internalModuleExamples\n\n // Order the results descending by the number of internal modules that use the external module\n ORDER BY numberOfInternalModules DESC, externalModuleName ASC"

```

	externalModuleName	numberOfInternalModules	sumNumberOfUsedExternalDeclarations	minNumberOfUsedExternalDeclarations	maxNumberOfUsedExternalDeclarations
0	@remix-run/router	4	131		6
1	@types/react	4	37		1
2	@types/react-native	1	10		10
3	@ungap/url-search-params	1	1		1

Table 3a - Top 20 most widely spread external packages - number of internal modules

This table shows the top 20 most widely spread external packages focussing on the spread across the number of internal modules.

	externalModuleName	numberOfInternalModules	minNumberOfInternalElements	maxNumberOfInternalElements	medNumberOfInternalElements
0	@remix-run/router	4	3		23
1	@types/react	4	1		24
2	@types/react-native	1	5		5
3	@ungap/url-search-params	1	2		2

Table 3b - Top 20 most widely spread external packages - percentage of internal modules

This table shows the top 20 most widely spread external packages focussing on the spread across the percentage of internal modules.

	externalModuleName	numberOfInternalModules	minNumberOfInternalElementsPercentage	maxNumberOfInternalElementsPercentage	medNumberOfInternalElementsPercentage
0	@remix-run/router	4	75.0		575.0
1	@types/react	4	25.0		600.0
2	@types/react-native	1	125.0		125.0
3	@ungap/url-search-params	1	50.0		50.0

Table 3c - Top 20 most widely spread external packages - number of internal elements

This table shows the top 20 most widely spread external packages focussing on the spread across the number of internal elements.

	externalModuleName	numberOfInternalModules	minNumberOfInternalElements	maxNumberOfInternalElements	medNumberOfInternalElements
0	@remix-run/router	4	3		23
1	@types/react	4	1		24
2	@types/react-native	1	5		5
3	@ungap/url-search-params	1	2		2

Table 3d - Top 20 most widely spread external packages - percentage of internal elements

This table shows the top 20 most widely spread external packages focussing on the spread across the percentage of internal elements.

	externalModuleName	numberOfInternalModules	minNumberOfInternalElementsPercentage	maxNumberOfInternalElementsPercentage	medN
0	@remix-run/router	4	75.0	575.0	
1	@types/react	4	25.0	600.0	
2	@types/react-native	1	125.0	125.0	
3	@ungap/url-search-params	1	50.0	50.0	

Table 3 Chart 1a - Most widely spread external module in % by internal elements (more than 0.5% overall)

External modules that are used less than 0.5% are grouped into the name "others" to get a cleaner chart with the most significant external module.

<Figure size 640x480 with 0 Axes>
Top external module usage spread [%] by internal elements (more than 0.5% overall)

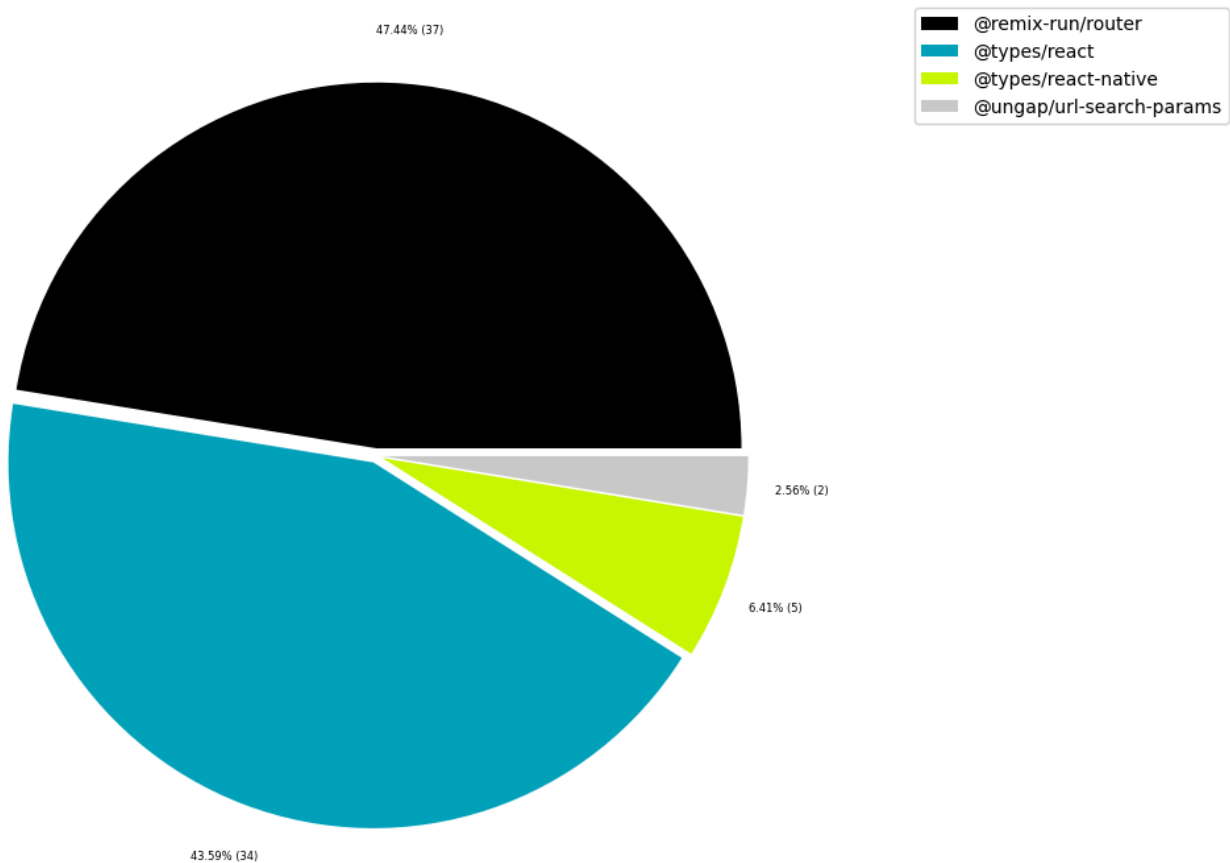


Table 3 Chart 1b - Most widely spread external modules in % by types (less than 0.5% overall "others" drill-down)

Shows the lowest (less than 0.5% overall) most widely spread external modules. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external module usage spread [%] by internal elements (less than 0.7% overall "others" drill-down)'.

Table 3 Chart 2a - Most widely spread external modules in % by internal modules (more than 0.5% overall)

External modules that are used less than 0.5% are grouped into "others" to get a cleaner chart containing the most significant external modules.

<Figure size 640x480 with 0 Axes>

Top external module usage spread [%] by internal modules (more than 0.5% overall)

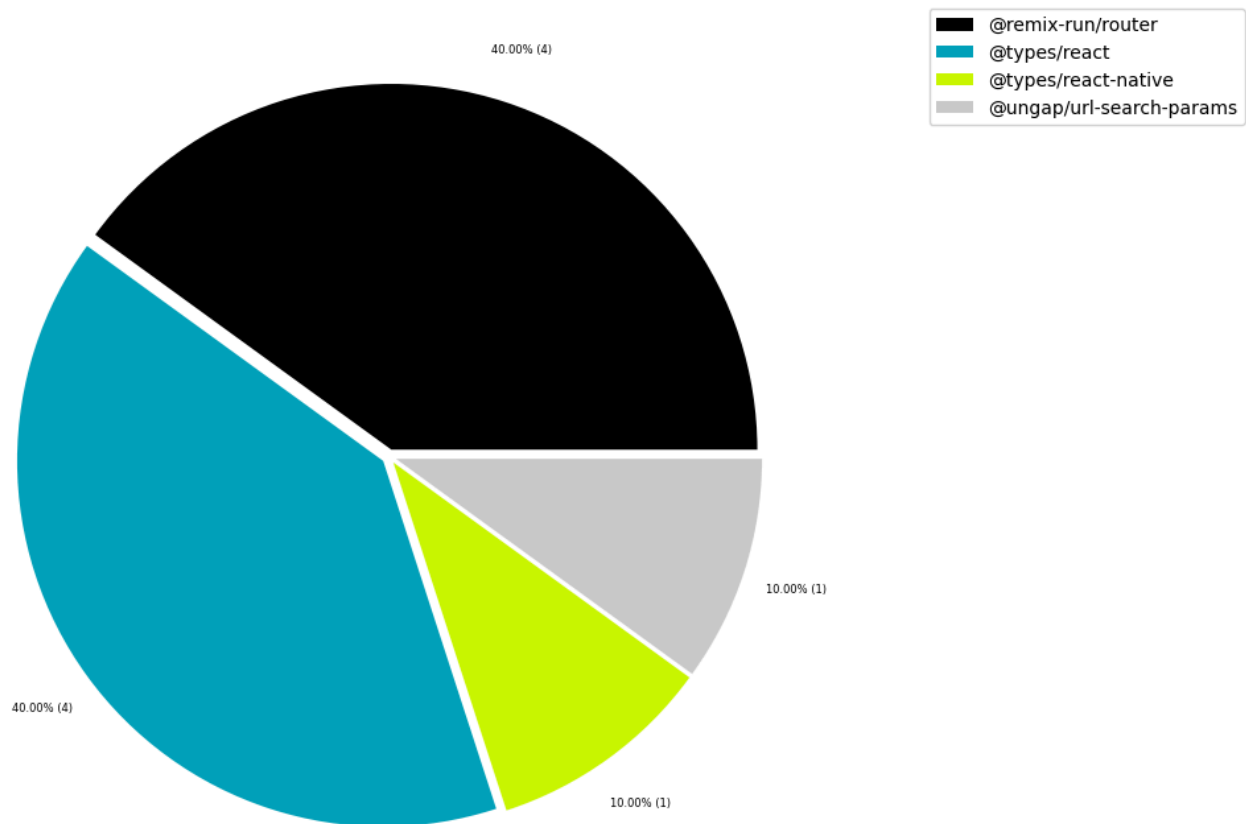


Table 3 Chart 2b - Most widely spread external modules in % by internal modules (less than 0.5% overall "others" drill-down)

Shows the lowest (less than 0.5% overall) most widely spread external modules. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external module usage spread [%] by internal modules (less than 0.7% overall "others" drill-down)'.

Table 4 - Top 20 most widely spread external namespaces

This table shows external namespaces that are used by different internal modules with the most used first.

Statistics like minimum, maximum, average, median and standard deviation are provided for the number of internally exported elements (function, class, ...) and the external declarations they use for every external namespace.

The intuition behind that is to find external namespaces that are used in a widely spread manner. This can help to distinguish widely used libraries and frameworks from external modules that are used for specific tasks. It can also be used to find external modules that are used sparsely regarding internal modules but where many different external declarations are used.

Refactoring with a [Hexagonal architecture](#) in mind can be considered for non-framework external namespaces that are used for very specific tasks and that are used in many different internal locations. This makes the internal code more robust against changes of these external modules or it is easier to update and migrate to newer versions of them.

External namespaces that are only used in very few internal locations overall might be considered for removal if they are easy to replace with a similar library that is already used more often. Or they might also simply be replaced by very few lines of code. Replacing libraries with own code isn't recommended when you need to write a lot of code or for external modules that provide security relevant implementations (encryption, sanitizers, ...), because they will be tracked and maintained globally and security updates need to be adopted fast.

Only the top 20 entries are shown. The whole table can be found in the following CSV report: `External_namespace_usage_spread_for_Typescript`

Columns:

- *externalModuleNamespace* identifies the external namespace for at least on external module in use. All other columns contain aggregated data for it.
- *numberOfInternalModules* is the number of internal modules that are using that external module
- *[min,max,med,avg,std]NumberOfUsedExternalDeclarations* provide statistics for all internal modules and how their usage of the declarations provided by the external module are distributed. This provides an indicator on how strong the coupling to the external module is. For example, if many (high sum) elements provided by that external module are used constantly (low std), a higher coupling can be presumed. If there is only one (sum) element in use, this could be an indicator for an external module that could get replaced or that there is just one central entry point for it.
- *[min/max/med/avg/std]NumberOfInternalElements* provide statistics for all internal modules and how their usage of the external module is distributed across their internal elements. This provides an indicator on how widely an external module is spread across

internal elements and if there are great differences between internal modules (high standard deviation) or not.

- *[min/max/med/avg/std]NumberOfInternalElementsPercentage* is similar to *[min/max/med/avg/std]NumberOfUsedExternalDeclarations* but provides the value in percent in relation to the total number of internal elements per internal module.
- *internalModuleExamples* some examples of included internal modules for debugging

Table 4 - Top 20 most widely spread external namespaces

This table shows external namespaces that are used by different internal modules with the most used first.

Statistics like minimum, maximum, average, median and standard deviation are provided for the number of internally exported elements (function, class, ...) and the external declarations they use for every external namespace.

The intuition behind that is to find external namespaces that are used in a widely spread manner. This can help to distinguish widely used libraries and frameworks from external modules that are used for specific tasks. It can also be used to find external modules that are used sparsely regarding internal modules but where many different external declarations are used.

Refactoring with a [Hexagonal architecture](#) in mind can be considered for non-framework external namespaces that are used for very specific tasks and that are used in many different internal locations. This makes the internal code more robust against changes of these external modules or it is easier to update and migrate to newer versions of them.

External namespaces that are only used in very few internal locations overall might be considered for removal if they are easy to replace with a similar library that is already used more often. Or they might also simply be replaced by very few lines of code. Replacing libraries with own code isn't recommended when you need to write a lot of code or for external modules that provide security relevant implementations (encryption, sanitizers, ...), because they will be tracked and maintained globally and security updates need to be adopted fast.

Only the top 20 entries are shown. The whole table can be found in the following CSV report: `External_namespace_usage_spread_for_Typecript`

Columns:

- *externalModuleNamespace* identifies the external namespace for at least on external module in use. All other columns contain aggregated data for it.
- *numberOfInternalModules* is the number of internal modules that are using that external module

- *[min,max,med,avg,std]NumberOfUsedExternalDeclarations* provide statistics for all internal modules and how their usage of the declarations provided by the external module are distributed. This provides an indicator on how strong the coupling to the external module is. For example, if many (high sum) elements provided by that external module are used constantly (low std), a higher coupling can be presumed. If there is only one (sum) element in use, this could be an indicator for an external module that could get replaced or that there is just one central entry point for it.
- *[min/max/med/avg/std]NumberOfInternalElements* provide statistics for all internal modules and how their usage of the external module is distributed across their internal elements. This provides an indicator on how widely an external module is spread across internal elements and if there are great differences between internal modules (high standard deviation) or not.
- *[min/max/med/avg/std]NumberOfInternalElementsPercentage* is similar to *[min/max/med/avg/std]NumberOfUsedExternalDeclarations* but provides the value in percent in relation to the total number of internal elements per internal module.
- *internalModuleExamples* some examples of included internal modules for debugging

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 5, column: 27, offset: 193} for query: "// External Typescript namespace usage spread\n\n // Get the overall internal modules statistics first\nMATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n WITH count(DISTINCT internalModule.globalFqn)\n AS internalModulesCountOverall\n ,count(DISTINCT internalElement.globalFqn)\n AS internalElementsCountOverall\n ,collect(DISTINCT internalElement)\n AS internalElementList\n\n // Get the external declarations for each internal element\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:TS:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH internalModulesCountOverall\n ,internalElementsCountOverall\n ,coalesce(nullif(externalModule.namespace, ''), 'no namespace') AS externalModuleNamespace\n ,coalesce(nullif(internalModule.namespace, '') + '/' + internalModule.name, internalModule.name) AS internalModuleName\n\n // Gathering counts for every internal element and the external module it uses\n ,count (DISTINCT externalDeclaration.globalFqn) AS externalDeclarationsCount\n ,COLLECT(DISTINCT externalDeclaration.globalFqn )[0..9] AS externalDeclarationsExamples\n ,count (DISTINCT internalElement.globalFqn) AS internalElementsCount\n ,COLLECT (DISTINCT internalElement.globalFqn )[0..9] AS internalElementsExamples\n ,100.0 / internalModulesCountOverall\n * count(DISTINCT internalElement.globalFqn) AS internalElementsCallingExternalRate\n\n // Group by external module namespace\n RETURN externalModuleNamespace\n ,count(DISTINCT internalModuleName) AS numberOfInternalModules\n\n // Statistics about how many internal modules are using that external module\n ,sum(externalDeclarationsCount) AS sumNumberOfUsedExternalDeclarations\n ,min(externalDeclarationsCount) AS minNumberOfUsedExternalDeclarations\n ,max(externalDeclarationsCount) AS maxNumberOfUsedExternalDeclarations\n ,percentileCont(externalDeclarationsCount, 0.5) AS medNumberOfUsedExternalDeclarations\n ,avg(externalDeclarationsCount) AS avgNumberOfUsedExternalDeclarations\n ,stDev(externalDeclarationsCount) AS stdNumberOfUsedExternalDeclarations\n\n // Statistics about the internal elements and their external module usage\n ,sum(internalElementsCount) AS sumNumberOfInternalElements\n ,min(internalElementsCount) AS minNumberOfInternalElements\n ,max(internalElementsCount) AS maxNumberOfInternalElements\n ,percentileCont(internalElementsCount, 0.5) AS medNumberOfInternalElements\n ,avg(internalElementsCount) AS avgNumberOfInternalElements\n ,stDev(internalElementsCount) AS stdNumberOfInternalElements\n\n // Statistics about the types and their external package usage count percentage\n ,min(internalElementsCallingExternalRate) AS minNumberOfInternalElementsPercentage\n ,max(internalElementsCallingExternalRate) AS maxNumberOfInternalElementsPercentage\n ,percentileCont(internalElementsCallingExternalRate, 0.5) AS medNumberOfInternalElementsPercentage\n ,avg(internalElementsCallingExternalRate) AS avgNumberOfInternalElementsPercentage\n ,stDev(internalElementsCallingExternalRate) AS stdNumberOfInternalElementsPercentage\n\n ,collect(DISTINCT internalModuleName)[0..4] AS internalModuleExamples\n\n\n // Order the results descending by the number of internal modules that use the external namespace\n ORDER BY numberOfInternalModules DESC, externalModuleNamespace ASC"

```

	externalModuleNamespace	numberOfInternalModules	sumNumberOfUsedExternalDeclarations	minNumberOfUsedExternalDeclarations	maxI
0	@remix-run	4	131		6
1	@types	4	47		1
2	@ungap	1	1		1

Table 4 Chart 1a - Most widely spread external namespaces in % by internal element (less than 0.5% overall)

External namespaces that are used less than 0.5% are grouped into "others" to get a cleaner chart containing the most significant external namespaces and how often they are called in percent.

<Figure size 640x480 with 0 Axes>

Top external namespace usage spread [%] by internal elements (less than 0.5% overall)

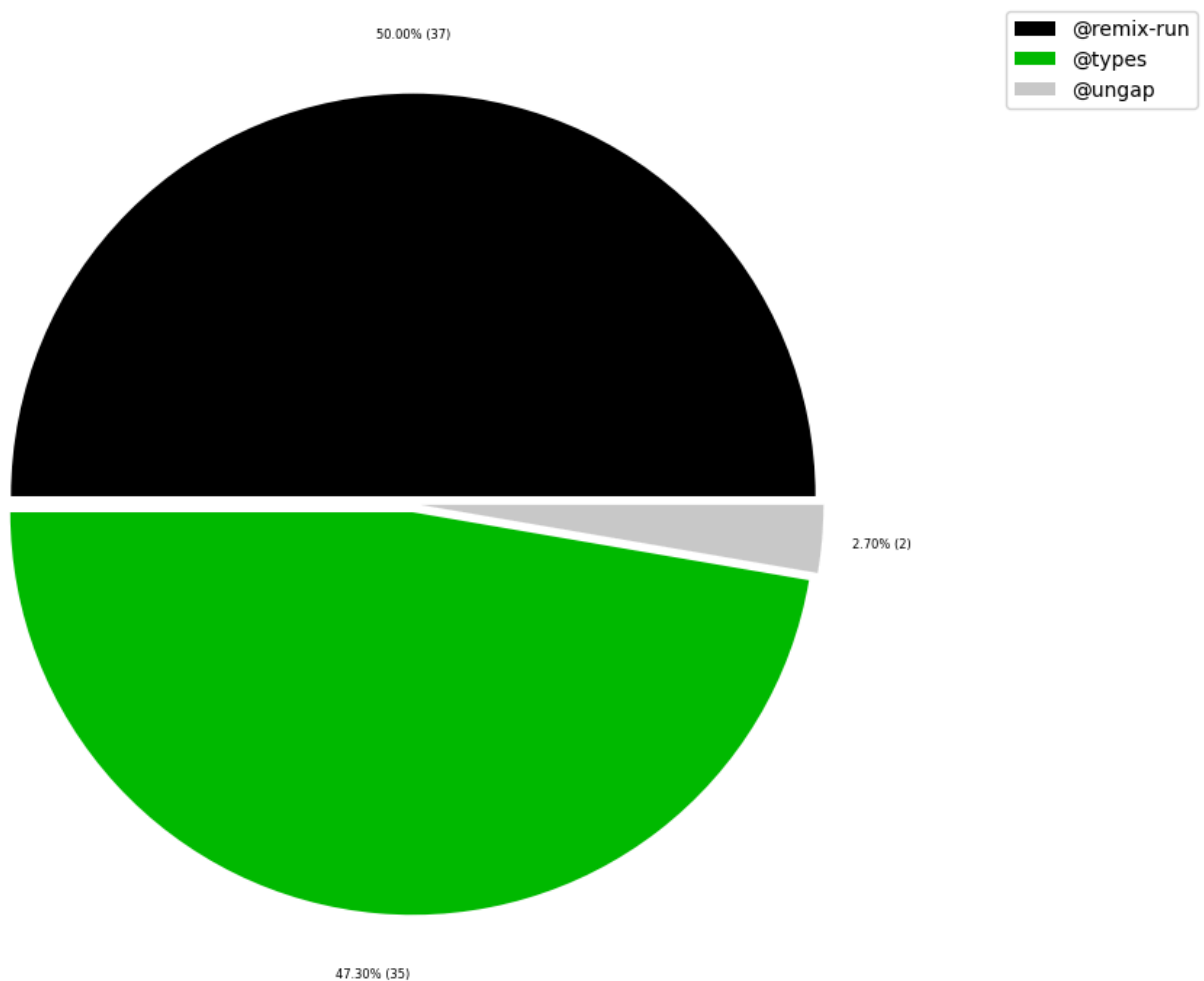


Table 4 Chart 1b - Most widely spread external namespaces in % by internal element (less than 0.5% overall "others" drill-down)

Shows the lowest (less than 0.5% overall) most widely spread external namespaces. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external namespace usage spread [%] by internal elements (less than 0.7% overall "others" drill-down)'.

Table 4 Chart 2a - Most widely spread external namespace in % by internal modules (more than 0.5% overall)

External namespaces that are used less than 0.5% are grouped into "others" to get a cleaner chart containing the most significant external namespaces and how often they are called in percent.

<Figure size 640x480 with 0 Axes>

Top external namespace usage spread [%] by internal modules (more than 0.5% overall)

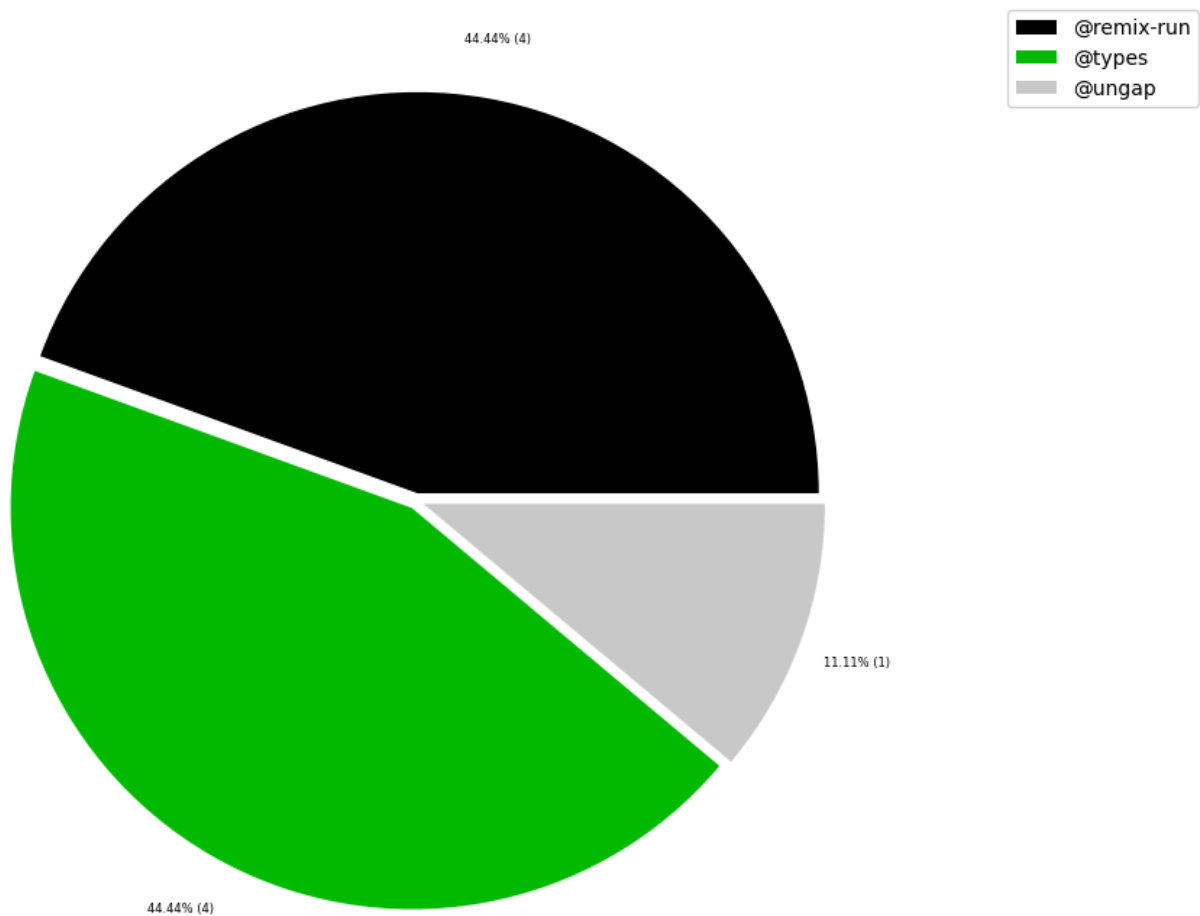


Table 4 Chart 2b - Most widely spread external namespace in % by internal modules (less than 0.5% overall "others" drill-down)

Shows the lowest (less than 0.5% overall) most widely spread external namespaces. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external namespace usage spread [%] by internal modules (less than 0.7% overall "others" drill-down)'.

Table 4 Chart 3a - External namespaces with the most used declarations in % (more than 0.5% overall)

External namespaces that are used less than 0.5% are grouped into "others" to get a cleaner chart containing the most significant external namespaces and how often they are called in percent.

<Figure size 640x480 with 0 Axes>

Top external namespace declaration usage [%] (more than 0.5% overall)

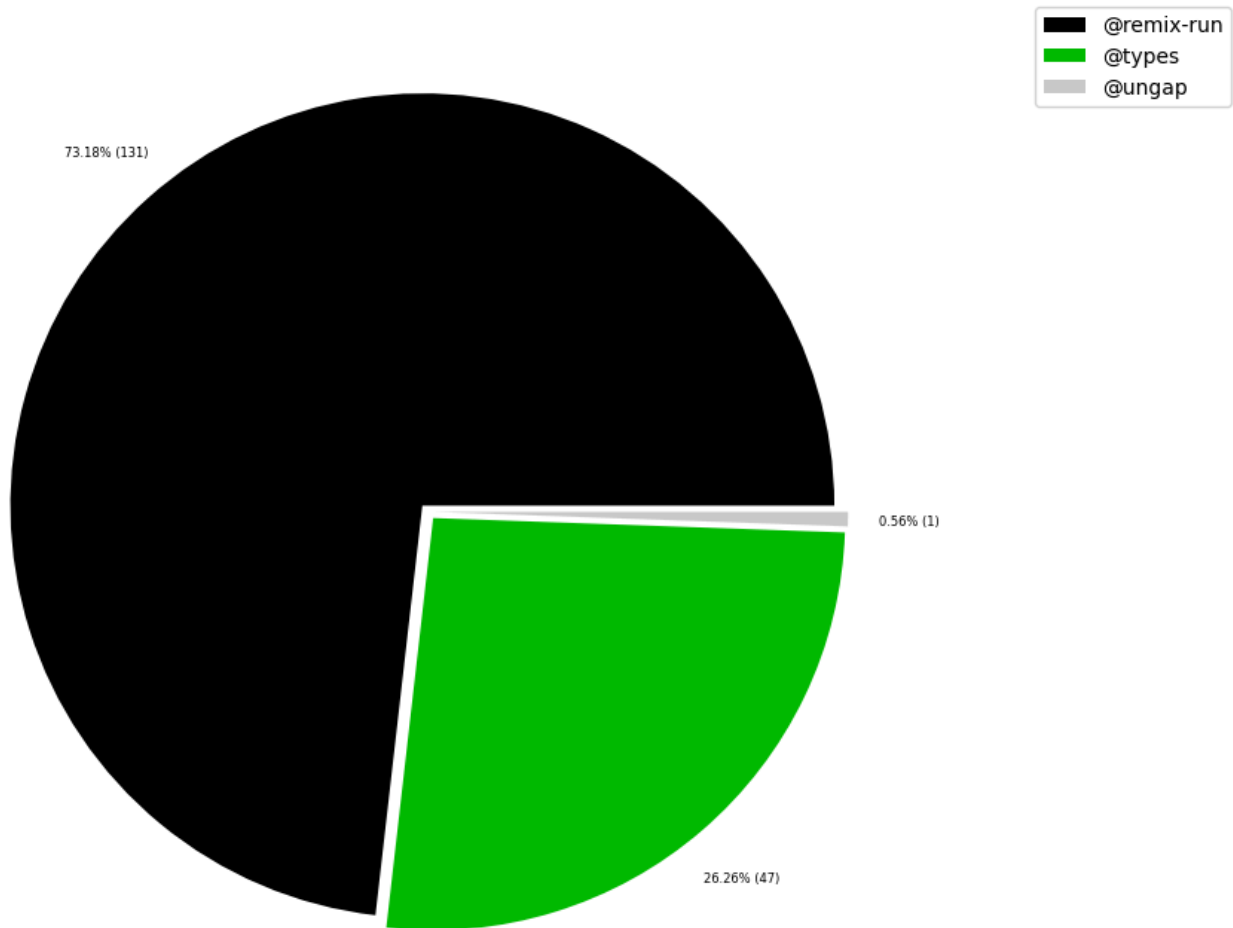


Table 4 Chart 3b - External namespaces with the most used declarations in % (less than 0.5% overall "others" drill-down)

Shows the lowest (less than 0.5% overall) external namespaces with the most used declarations. Therefore, this plot breaks down the "others" slice of the pie chart above. Values under 0.3% from that will be grouped into "others" to get a cleaner plot.

No data to plot for title 'Top external namespace declaration usage (less than 0.7% overall "others" drill-down)'.

Table 5 - Top 20 least used external modules overall

This table identifies external modules that aren't used very often. This could help to find libraries that aren't actually needed or maybe easily replaceable. Some of them might be used sparsely on purpose for example as an adapter to an external library that is actually important. Thus, decisions need to be made on a case-by-case basis.

Only the last 20 entries are shown. The whole table can be found in the following CSV report: `External_module_usage_overall_for_Typescript`

Columns:

- *externalModuleName* identifies the external package as described above
- *numberOfExternalDeclarationCalls* includes every invocation or reference to the declarations in the external module

	externalModuleName	numberOfExternalDeclarationCalls
0	@ungap/url-search-params	2
1	@types/react-native	14
2	@types/react	97
3	@remix-run/router	195

Table 6 - External usage per internal module sorted by highest external element usage rate descending

The following table shows the most used external packages separately for each artifact including external annotations. The results are sorted by the artifacts with the highest external type usage rate descending.

The intention of this table is to find artifacts that use a lot of external dependencies in relation to their size and get all the external packages and their usage.

Only the first 40 entries are shown. The whole table can be found in the following CSV report: `External_module_usage_per_internal_module_sorted_for_Typescript`

Columns:

- *internalModuleName* is the internal module that uses the external one. Both are used here as a group for a more detailed analysis.

- *externalModuleName* is the external module prepended by its namespace if given.
Example: "@types/react"
- *numberOfExternalDeclarationCaller* is the count of distinct internal elements in the internal module that call the external module
- *numberOfExternalDeclarationCalls* is the count of how often the external module is called within the internal module
- *numberOfAllElementsInInternalModule* is the total count of all exported elements of the internal module
- *numberOfAllExternalDeclarationsUsedInInternalModule* is the total count of all distinct external declarations used in the internal module
- *numberOfAllExternalModulesUsedInInternalModule* is the total count of all distinct external modules used in the internal module
- *externalDeclarationRate* is the $\text{numberOfAllExternalDeclarationsUsedInInternalModule} / \text{numberOfAllElementsInInternalModule} * 100$ of the internal module for all external modules
- *externalDeclarationNames* contains a list of actually used external declarations

Table 6a - External module usage per internal module sorted by highest external element usage rate descending

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 4, column: 27, offset: 186} for query: "// External Typescript module usage per internal module sorted by external usage descending\n\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n OPTIONAL MATCH (internalElement)-[:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n\n WHERE externalDeclaration.isExternalImport = true\n\n OPTIONAL MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n\n WITH internalModule.name AS internalModuleName\n, count(DISTINCT internalElement.globalFqn) AS numberOfAllElementsInInternalModule\n\n, count(DISTINCT externalDeclaration.globalFqn) AS numberOfAllExternalDeclarationsUsedInInternalModule\n\n, count(DISTINCT externalModule.globalFqn) AS numberOfAllExternalModulesUsedInInternalModule\n\n, collect(DISTINCT internalElement)\n AS internalElementList\n UNWIND internalElementList AS internalElement\n\n MATCH (internalElement)-[externalDependency:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n\n WITH numberOfAllElementsInInternalModule\n\n, numberOfAllExternalDeclarationsUsedInInternalModule\n\n, numberOfAllExternalModulesUsedInInternalModule\n\n, 100.0 / numberOfAllElementsInInternalModule * numberOfAllExternalDeclarationsUsedInInternalModule AS externalDeclarationRate\n\n, externalDependency\n\n, internalModuleName\n\n, internalElement.globalFqn AS fullInternalElementName\n\n, internalElement.name AS internalElementName\n\n, coalesce(\n\n nullIf(externalModule.namespace, '') + '/' + externalModule.name, \n\n externalModule.name) AS externalModuleName\n\n, externalDeclaration.name AS externalDeclarationName\n\n WITH numberOfAllElementsInInternalModule\n\n, numberOfAllExternalDeclarationsUsedInInternalModule\n\n, numberOfAllExternalModulesUsedInInternalModule\n\n, externalDeclarationRate\n\n, internalModuleName\n\n, externalModuleName\n\n, count(externalDependency) AS numberOfExternalDeclarationCaller\n\n, sum(externalDependency.cardinality) AS numberOfExternalDeclarationCalls\n\n, collect(DISTINCT externalDeclarationName) AS externalDeclarationNames\n\n RETURN internalModuleName\n\n, externalModuleName\n\n, numberOfExternalDeclarationCaller\n\n, numberOfExternalDeclarationCalls\n\n, numberOfAllElementsInInternalModule\n\n, numberOfAllExternalDeclarationsUsedInInternalModule\n\n, numberOfAllExternalModulesUsedInInternalModule\n\n, externalDeclarationRate\n\n, externalDeclarationNames\n\n ORDER BY externalDeclarationRate DESC\n\n, internalModuleName ASC\n\n, numberOfExternalDeclarationCaller DESC\n\n, externalModuleName ASC"

```

	internalModuleName	externalModuleName	numberOfExternalDeclarationCaller	numberOfExternalDeclarationCalls	numberOfAllElementsInIn
0	server	@remix-run/router	37	48	
1	server	@types/react	3	7	
2	react-router-dom	@remix-run/router	141	217	
3	react-router-dom	@types/react	83	168	
4	react-router-native	@types/react-native	11	17	
5	react-router-native	@remix-run/router	9	13	
6	react-router-native	@types/react	9	17	
7	react-router-native	@ungap/url-search-params	2	4	
8	react-router	@remix-run/router	8	10	
9	react-router	@types/react	1	3	

Table 6b - External namespace usage per internal module sorted by highest external element usage rate descending

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 4, column: 27, offset: 186} for query: "// External Typescript module usage per internal module sorted by external usage descending\n\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n OPTIONAL MATCH (internalElement)-[:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n OPTIONAL MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH internalModule.name AS internalModuleName\n ,count(DISTINCT internalElement.globalFqn) AS numberOfAllElementsInInternalModule\n ,count(DISTINCT externalDeclaration.globalFqn) AS numberOfAllExternalDeclarationsUsedInInternalModule\n ,count(DISTINCT externalModule.globalFqn) AS numberOfAllExternalModulesUsedInInternalModule\n ,collect(DISTINCT internalElement) AS internalElementList\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[externalDependency:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WITH numberOfAllElementsInInternalModule\n ,numberOfAllExternalDeclarationsUsedInInternalModule\n ,numberOfAllExternalModulesUsedInInternalModule\n ,100.0 / numberOfAllElementsInInternalModule * numberOfAllExternalDeclarationsUsedInInternalModule AS externalDeclarationRate\n ,externalDependency\n ,internalModuleName\n ,internalElement.globalFqn AS fullInternalElementName\n ,internalElement.name AS internalElementName\n ,coalesce(nullif(externalModule.namespace, ''), 'no namespace') AS externalNamespaceName\n ,externalDeclaration.name AS externalDeclarationName\n WITH numberOfAllElementsInInternalModule\n ,numberOfAllExternalDeclarationsUsedInInternalModule\n ,numberOfAllExternalModulesUsedInInternalModule\n ,externalDeclarationRate\n ,internalModuleName\n ,externalNamespaceName\n ,count(externalDependency) AS numberOfExternalDeclarationCaller\n ,sum(externalDependency.cardinality) AS numberOfExternalDeclarationCalls\n ,collect(DISTINCT externalDeclarationName) AS externalDeclarationNames\n RETURN internalModuleName\n ,externalNamespaceName\n ,numberOfExternalDeclarationCaller\n ,numberOfExternalDeclarationCalls\n ,numberOfAllElementsInInternalModule\n ,numberOfAllExternalDeclarationsUsedInInternalModule\n ,numberOfAllExternalModulesUsedInInternalModule\n ,externalDeclarationRate\n ,externalDeclarationNames\n ORDER BY externalDeclarationRate DESC\n ,internalModuleName ASC\n ,numberOfExternalDeclarationCaller DESC\n ,externalNamespaceName ASC"

```

	internalModuleName	externalNamespaceName	numberOfExternalDeclarationCaller	numberOfExternalDeclarationCalls	numberOfAllElements
0	server	@remix-run	37	48	
1	server	@types	3	7	
2	react-router-dom	@remix-run	141	217	
3	react-router-dom	@types	83	168	
4	react-router-native	@types	20	34	
5	react-router-native	@remix-run	9	13	
6	react-router-native	@ungap	2	4	
7	react-router	@remix-run	8	10	
8	react-router	@types	1	3	

Table 6c - Top 15 used external modules with the internal modules that use them the most

The following table uses pivot to show the internal modules in columns, the external modules in rows and the number of internal elements using them as values.

internalModuleName	react-router-dom	server	react-router-native	react-router
externalModuleName				
@remix-run/router	141	37	9	8
@types/react	83	3	9	1
@types/react-native	0	0	11	0
@ungap/url-search-params	0	0	2	0

Table 6d - Top 15 used external namespaces with the internal modules that use them the most

The following table uses pivot to show the internal modules in columns, the external namespaces in rows and the number of internal elements using them as values.

internalModuleName	react-router-dom	server	react-router-native	react-router
externalNamespaceName				
@remix-run	141	37	9	8
@types	83	3	20	1
@ungap	0	0	2	0

Table 6e - External usage per internal module and its elements

This table lists internal elements and the modules they belong to that use many different external declarations of a specific external module.

	internalModuleName	numberOfAllExternalDeclarationsUsedInInternalModule	numberOfAllElementsInInternalModule
0	server	29	6
2	react-router-dom	117	63
4	react-router-native	24	17
8	react-router	9	7

Table 6 Chart 1 - Top 15 external dependency using artifacts and their external packages stacked

The following chart shows the top 15 external package using artifacts and breaks down which external packages they use in how many different internal packages with stacked bars.

Note that every external dependency is counted separately so that if on internal package uses two external packages it will be displayed for both and so stacked twice.

<Figure size 640x480 with 0 Axes>

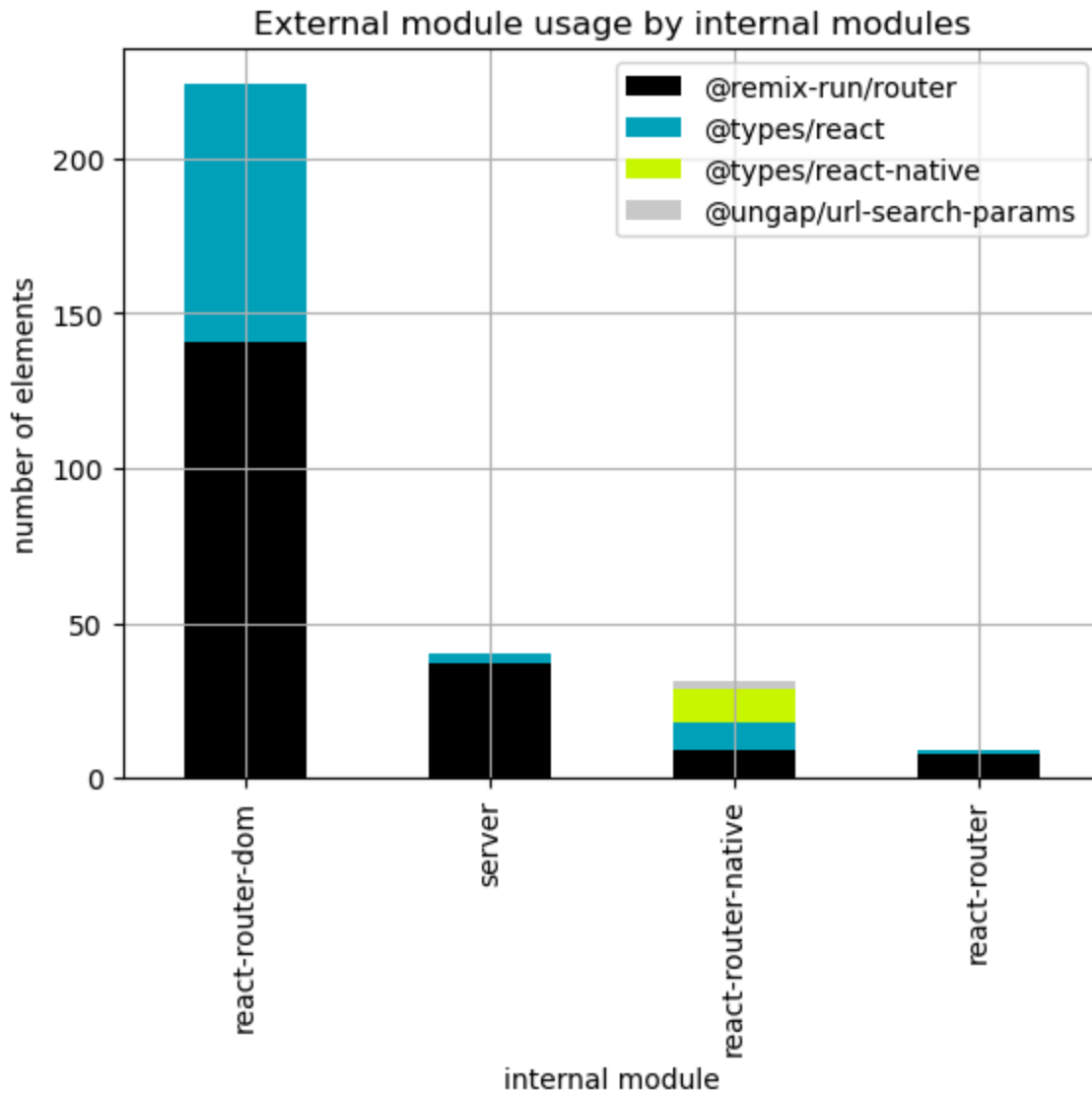


Table 6 Chart 2 - Top 15 external dependency using artifacts and their external packages (first 2 levels) stacked

The following chart shows the top 15 external package using artifacts and breaks down which external packages (first 2 levels) are used in how many different internal packages with stacked bars.

Note that every external dependency is counted separately so that if on internal package uses two external packages it will be displayed for both and so stacked twice.

<Figure size 640x480 with 0 Axes>

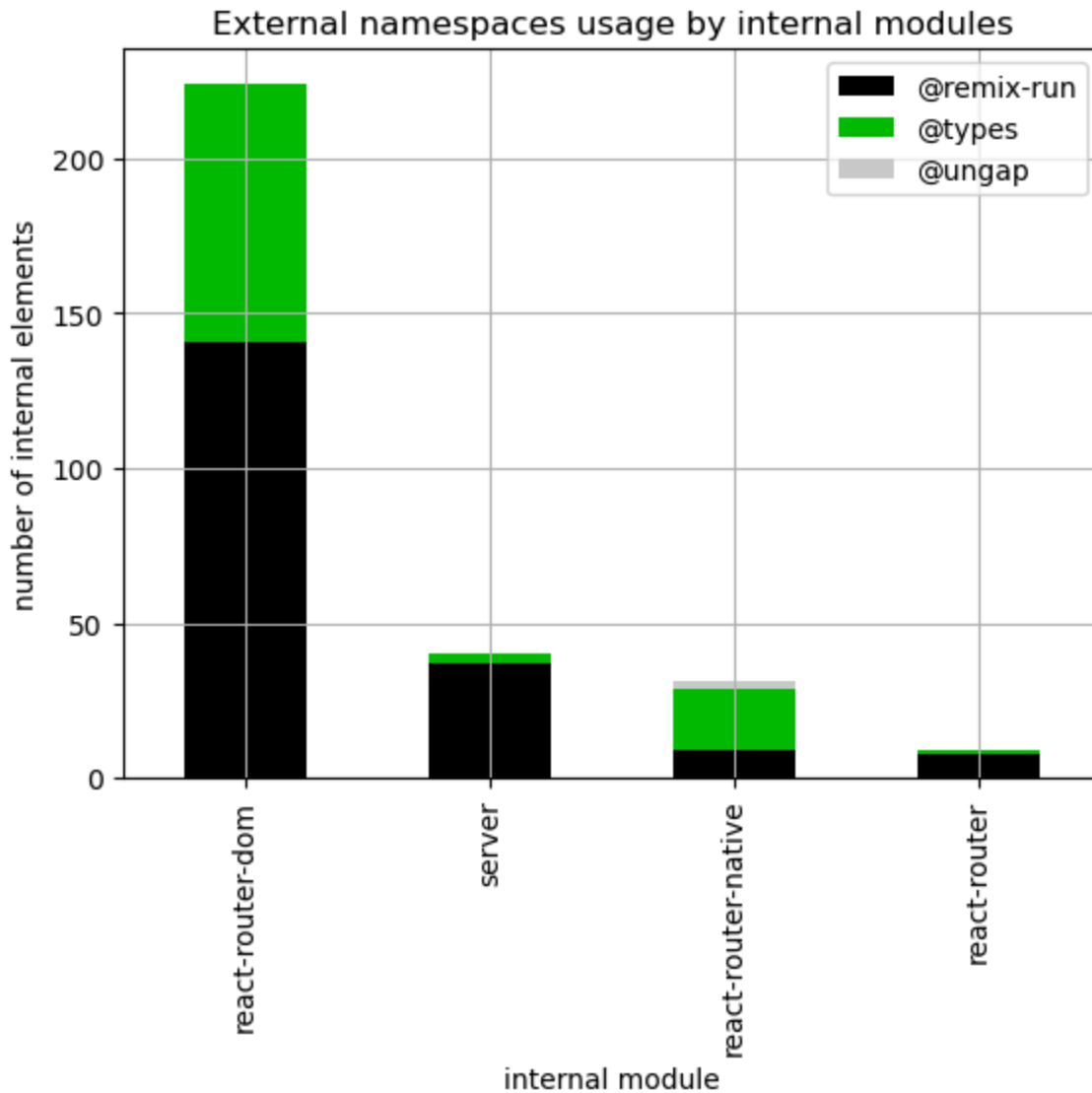


Table 7 - External module usage distribution per internal element

This table shows how many internal elements use one external module, how many use two, etc. . This gives an overview of the distribution of external module calls and the overall coupling to external libraries. The higher the count of distinct external modules the lower should be the count of internal elements that use them. More details about which types have the highest external package dependency usage can be in the tables 4 and 5 above.

Only the last 40 entries are shown. The whole table can be found in the following CSV report: [External_module_usage_per_internal_module_distribution_for_Typescript](#)

Columns:

- *internalModuleName* is the internal module that uses at least one external module. All other columns refer to it.

- *numberOfAllInternalElements* the total number of all elements that are exported by the internal module
- *externalModuleCount* is the number of distinct external modules used by the internal module
- *internalElementCount* is the number of distinct internal elements that use at least one external one
- *internalElementsCallingExternalRate* is $\text{internalElementCount} / \text{numberOfAllInternalElements} * 100$ (in %)

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 4, column: 27, offset: 164} for query: "// External Typescript module usage distribution for internal modules\n\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n WHERE NOT internalModule:TestRelated\n WITH internalModule.name AS internalModuleName\n ,count(DISTINCT internalElement.globalFqn) AS numberOfAllInternalElements\n ,collect(DISTINCT internalElement) AS internalElementList\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[:DEPENDS_ON]->(externalElement:ExternalDeclaration)\n WHERE externalElement.isExternalImport = true\n MATCH (internalModule:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalElement)\n WITH internalModuleName\n ,numberOfAllInternalElements\n ,internalModule.globalFqn AS fullInternalModuleName\n ,internalElement.globalFqn AS fullInternalElementName\n ,coalesce(\n nullIf\n (externalModule.namespace, '') + '/' + externalModule.name, \n externalModule.name) AS externalModuleName\n WITH internalModuleName\n ,numberOfAllInternalElements\n ,count(DISTINCT externalModuleName) AS externalModuleCount\n ,COLLECT(DISTINCT externalModuleName)[0..9] AS externalModuleExamples\n ,count(DISTINCT fullInternalElementName) AS internalElementCount\n ,COLLECT(DISTINCT fullInternalElementName)[0..9] AS internalElementExamples\n RETURN internalModuleName\n ,numberOfAllInternalElements\n ,externalModuleCount\n ,internalElementCount\n ,100.0 / numberOfAllInternalElements * internalElementCount AS internalElementsCallingExternalRate\n ,externalModuleExamples\n ,internalElementExamples\n ORDER BY internalElementCount DESC, internalModuleName ASC"
```

	internalModuleName	numberOfAllInternalElements	externalModuleCount	internalElementCount	internalElementsCallingExternalRate
0	react-router-dom	63	2	29	46.031746
1	react-router-native	17	4	10	58.823529
2	server	6	2	6	100.000000
3	react-router	7	2	3	42.857143

Table 8 - External package usage aggregated

This table lists all artifacts and their external package dependencies usage aggregated over internal packages.

The intention behind this is to find artifacts that use an external dependency across multiple internal packages. This might be intended for frameworks and standardized

libraries and helps to quantify how widely those are used. For some external dependencies it might be beneficial to only access it from one package and provide an abstraction for internal usage following a [Hexagonal architecture](#). Thus, this table may also help in finding application for the Hexagonal architecture or similar approaches (Domain Driven Design Anti Corruption Layer). After all it is easier to update or replace such external dependencies when they are used in specific areas and not all over the code.

Only the last 40 entries are shown. The whole table can be found in the following CSV report: `External_module_usage_per_internal_module_aggregated_for_Typescript`

Columns:

- *internalModuleName* that contains the type that calls the external package
- *internalModuleElementsCount* is the total count of packages in the internal module
- *numberOfExternalModules* the number of distinct external packages used
- *[min,max,med,avg,std]NumberOfInternalModules* provide statistics based on each external package and its package usage within the internal module
- *[min,max,med,avg,std]NumberOfInternalElements* provide statistics based on each external package and its type usage within the internal module
- *[min,max,med,avg,std]NumberOfTypePercentage* provide statistics in % based on each external package and its type usage within the internal module in respect to the overall count of packages in the internal module
- *numberOfInternalElements* in the internal module where the *numberOfExternalModules* applies
- *numberOfTypesPercentage* in the internal module where the *numberOfExternalModules* applies in %

Table 8a - External module usage aggregated - count of internal modules

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: TestRelated)} {position: line: 5, column: 27, offset: 213} for query: "// External Typescript module usage per internal module aggregated\n\n // Get the overall internal module statistics first\n MATCH (internalModule:TS:Module)-[:EXPORTS]->(internalElement:TS)\n\n WHERE NOT internalModule:TestRelated\n WITH internalModule.name\n AS internalModuleName\n ,internalModule.communityLeidenId\n AS leidenCommunityId\n ,count(DISTINCT internalElement.globalFqn) AS internalModuleElementsCount\n ,collect(DISTINCT internalElement) AS internalElementList\n\n // Get the external dependencies for each internal internalElement\n UNWIND internalElementList AS internalElement\n MATCH (internalElement)-[:DEPENDS_ON]->(externalDeclaration:ExternalDeclaration)\n MATCH (internalModule:Module)-[:EXPORTS]->(internalElement)\n MATCH (externalModule:ExternalModule)-[:EXPORTS]->(externalDeclaration)\n WHERE externalDeclaration.isExternalImport = true\n WITH internalModuleName\n ,leidenCommunityId\n ,internalModuleElementsCount\n ,internalModule.globalFqn\n AS fullInternalModuleName\n ,internalElement.globalFqn\n AS fullInternalElementName\n ,coalesce(\n\n nullIf(externalModule.namespace, '') +\n '/' + externalModule.name, \n\n externalModule.name) AS externalModuleName\n\n\n // Group by internalModule and external internalElement\n WITH internalModuleName\n ,leidenCommunityId\n ,internalModuleElementsCount\n ,externalModuleName\n ,count(DISTINCT fullInternalModuleName) AS internalModulesCount\n ,COLLECT(DISTINCT fullInternalModuleName)[0..9] AS internalModulesExamples\n ,count(DISTINCT fullInternalElementName) AS internalElementsCount\n ,COLLECT(DISTINCT fullInternalElementName)[0..9] AS internalElementsExamples\n ,100.0 / internalModuleElementsCount * count(DISTINCT fullInternalElementName) AS internalElementsCallingExternalRate\n\n\n // Pre order the results by number of packages that use the external package dependency descending\n ORDER BY internalModulesCount DESC, internalModuleName ASC\n\n\n // Optionally filter out external dependencies that are only used by one internal module\n // WHERE internalModulesCount > 1\n\n\n // Group by internalModule, aggregate statistics and return the results\n RETURN internalModuleName\n ,leidenCommunityId\n ,internalModuleElementsCount\n\n ,count(DISTINCT externalModuleName) AS numberOfExternalModules\n\n\n // Statistics about the packages and their external package usage count\n ,min(internalModulesCount) AS minNumberOfInternalModules\n ,max(internalModulesCount) AS maxNumberOfInternalModules\n ,percentileCont(internalModulesCount, 0.5) AS medNumberOfInternalModules\n ,avg(internalModulesCount) AS avgNumberOfInternalModules\n ,stdDev(internalModulesCount) AS stdNumberOfInternalModules\n\n\n // Statistics about the types and their external package usage count\n ,min(internalElementsCount) AS minNumberOfInternalElements\n ,max(internalElementsCount) AS maxNumberOfInternalElements\n ,percentileCont(internalElementsCount, 0.5) AS medNumberOfInternalElements\n ,avg(internalElementsCount) AS avgNumberOfInternalElements\n ,stdDev(internalElementsCount) AS stdNumberOfInternalElements\n\n\n // Statistics about the types and their external package usage count percentage\n ,min(internalElementsCallingExternalRate) AS minNumberOfInternalElementsPercentage\n ,max(internalElementsCallingExternalRate) AS maxNumberOfInternalElementsPercentage\n ,percentileCont(internalElementsCallingExternalRate, 0.5) AS medNumberOfInternalElementsPercentage\n ,avg(internalElementsCallingExternalRate) AS avgNumberOfInternalElementsPercentage\n ,stdDev(internalElementsCallingExternalRate) AS stdNumberOfInternalElementsPercentage\n\n\n // Examples of external packages, caller packages and caller types\n ,collect(externalModuleName)[0..9]

```

```
AS top10ExternalPackageNamesByUsageDescending\n      ,COLLECT(internalModulesExamples)[0]
      AS internalModulesExamples\n      ,COLLECT(internalElementsExamples)[0]
      AS internalElementsExamples\n \n ORDER BY maxNumberOfInternalModules DESC, internalModuleName ASC"
```

	internalModuleName	internalModuleElementsCount	numberOfExternalModules	minNumberOfInternalModules	medNumberOfInternalModules
0	react-router	7	2	1	1.0
1	react-router-dom	63	2	1	1.0
2	react-router-native	17	4	1	1.0
3	server	6	2	1	1.0

Table 8b - External module usage aggregated - count of internal elements

	internalModuleName	internalModuleElementsCount	numberOfExternalModules	minNumberOfInternalElements	medNumberOfInternalElements
0	react-router	7	2	1	2
1	react-router-dom	63	2	23	23
2	react-router-native	17	4	2	5
3	server	6	2	3	4

Table 8c - External module usage aggregated - percentage of internal elements

	internalModuleName	internalModuleElementsCount	numberOfExternalModules	minNumberOfInternalElementsPercentage	medNumberOfInternalElements
0	react-router	7	2	14.285714	
1	react-router-dom	63	2	36.507937	
2	react-router-native	17	4	11.764706	
3	server	6	2	50.000000	

Table 8 Chart 1 - External module usage - max percentage of internal types

This chart shows per internal module the maximum percentage of internal packages (compared to all packages in that internal module) that use one specific external package.

Example: One internal module might use 10 external packages where 7 of them are used in one internal package, 2 of them are used in two packages and one external dependency is used in 5 packages. So for this internal module there will be a point at $x = 10$ (external packages used by the internal module) and 5 (max internal packages). Instead of the count the percentage of internal packages compared to all packages in that internal module is used to get a normalized plot.

<Figure size 640x480 with 0 Axes>

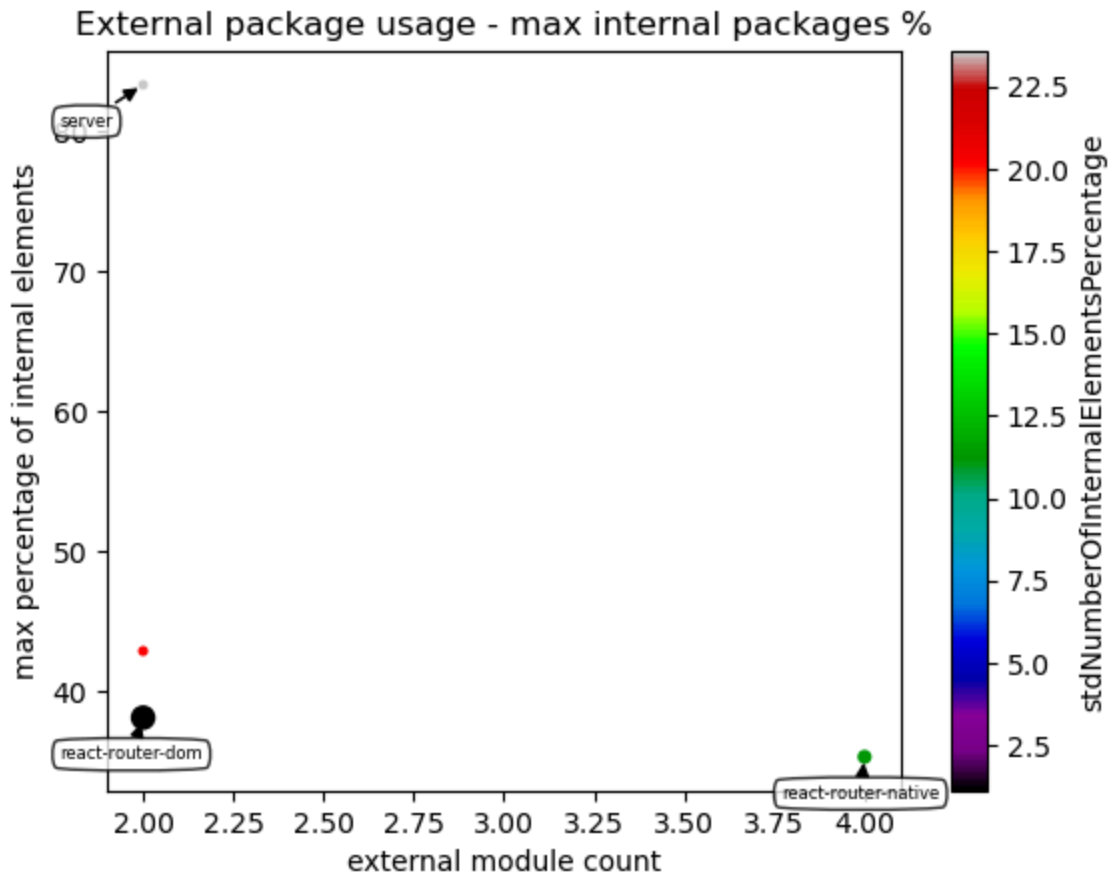


Table 8 Chart 2 - External package usage - median percentage of internal types

This chart shows per internal module the median (0.5 percentile) of internal packages (compared to all packages in that internal module) that use one specific external package.

Example: One internal module might use 9 external packages where 3 of them are used in 1 internal package, 3 of them are used in 2 package and the last 3 ones are used in 3 packages. So for this internal module there will be a point at $x = 10$ (external packages used by the internal module) and 2 (median internal packages). Instead of the count the percentage of internal packages compared to all packages in that internal module is used to get a normalized plot.

<Figure size 640x480 with 0 Axes>

External package usage - median internal packages %

